



Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Informe de Manejador de recursos distribuidos para HPC

Estudiantes: **Franco Bramucci,**
Ayrton Cuffaro,
Agustin Jaffre,
Lucas Lamberti

Docentes: **Taihú Pire,**
Erica Vidal,
Alejandro Rivosecchi,
Tomás Castro Rojas

Rosario, Junio, 2026

1. Organización del Equipo y Roles

La división de trabajos del Manejador de recursos distribuidos para HPC:

- **Rol 1 (Franco Bramucci):** Responsable del servidor C con `epoll`, gestión de conexiones entrantes/salientes, lectura/escritura no bloqueante, manejo de buffers y detección de cierres de sockets.
- **Rol 2 (Ayrton Cuffaro):** Diseño de la lógica de recursos locales (estructuras, colas FIFO, asignación y liberación), tabla de jobs activos y tabla de nodos.
- **Rol 3 (Lucas Lamberti):** Desarrollo del proceso en Erlang, conexión local al agente C, generación dinámica de jobs.
- **Rol 4 (Agustin Jaffre):** Estrategia de prevención/detección de deadlock, diagramas de secuencia de la comunicación Erlang-C, script de prueba automatizado.

Asimismo, se emplearon metodologías ágiles mediante reuniones de sincronización y control de versiones con Git utilizando la metodología de ramas por rol (excepto rol 3 y 4) para su posterior fusión.

2. Arquitectura del Sistema y Diseño de Software

2.1. Arquitectura General y Flujo de Control

El sistema adopta una arquitectura híbrida orientada a eventos y paso de mensajes distribuido. El Agente C (`server.c`) expone un socket de escucha local (`127.0.0.2:12000`) exclusivo para el planificador Erlang, y un socket público en el mismo puerto para la comunicación inter-agente.

Para optimizar el uso de recursos y evitar el bloqueo de hilos por conexión, el Agente C centraliza todos sus descriptores de archivo (sockets de escucha, de clientes y temporizadores) en una única instancia de `epoll`. Los sockets pasivos operan de forma no bloqueante (`SOCK_NONBLOCK`), mientras que los descriptores de los clientes se registran con la bandera `EPOLLONESHOT`, garantizando la exclusión mutua a nivel de kernel para que un único hilo procese a un cliente a la vez.

Al iniciar, el agente despliega un pool de hilos trabajadores (`pthread_t`) equivalente a la cantidad de núcleos del procesador, los cuales compiten de forma segura en una llamada bloqueante a `epoll_wait`. Ante una ráfaga de datos, el kernel despierta a un único hilo disponible que invoca a `leer_y_procesar_cliente()`. Para mitigar la fragmentación inherente al protocolo TCP, los datos se vuelcan sin bloqueo sobre un búfer estático gestionado dentro de una estructura persistente por conexión (`ClienteConexion`). Al detectarse una línea ASCII completa (`\n`), se delega el flujo a la función `manejar_cliente_erlang()`.

Por su parte, el planificador Erlang configura el socket TCP en modo pasivo (`{active, false}`), delegando la interacción de red al módulo `send_recv_manager.erl`, el cual desacopla la lectura y escritura en dos actores concurrentes:

- `esperar_respuesta_nodo`: Bloqueado permanentemente en `gen_tcp:recv`, actúa de forma simétrica al lector `epoll`. Al recibir y parsear comandos (ej.

"JOB_GRANTED 42\n"), despacha la información de manera asíncrona hacia el servidor de trabajos (job_server) liberando el canal de red inmediatamente.

- **send_manager:** Centraliza y serializa de forma segura las peticiones concurrentes del clúster (como jobRequest o jobRelease), formateando las cadenas ASCII con el delimitador \n obligatorio y empujándolas al flujo binario mediante gen_tcp:send.

Finalmente, dado que el Agente C opera únicamente con identificadores numéricos planos (JobId), el proceso job_server.erl actúa como puente de traducción hacia la máquina virtual BEAM. Este servidor mantiene un diccionario dinámico (JobMap) que mapea cada JobId entero con el PID del proceso ligero que ejecuta dicho trabajo. Sus primitivas fundamentales permiten registrar nuevos procesos (add), redirigir asíncronamente los cambios de estado notificados por la red (find), y purgar los registros al concluir (remove), notificando al orquestador raíz (all_jobsdone) una vez que el mapa se vacía.

2.2. Estructuras de Datos Internas en el Agente C

Descripción técnica de las estructuras en C:

- **Tabla de Nodos Activos:** La información que se guarda sobre cada nodo consiste en la ip, puerto y recursos que fueron recibidos en el ANNOUNCE junto a un timestamp que representa el momento en el que se haya recibido el anuncio.

Cada nodo pertenece a una tabla hash indexada por la ip del nodo lo que permite que la inserción de nodos nuevos descubiertos por el recibimiento de un ANNOUNCE sea eficiente junto a la actualización de sus timestamps en caso de que ya se hayan anunciado anteriormente.

Además, cada nodo pertenece a su vez a una lista doblemente enlazada para que de esta forma se pueda recorrer todos los nodos guardados en tiempo lineal (sin pasar por casillas vacías de la tabla hash. Esto último permite que sea eficiente obtener la información de todos los nodos guardados en la tabla (para la solicitud GET_NODES) y para que el thread encargado de la limpieza de nodos que no se hayan anunciado en el suficiente tiempo solo tenga que recorrer la lista, simplificando la el trabajo de este.

- **Tabla jobs:** Para cada job se guardará su job id, un identificador para diferenciar si el job se trata de un job local (proveniente de una solicitud de job del erlang) o un job remoto (las reservas de recursos locales recibidas de un nodo remoto) y los recursos reservados y pedidos del job.

Esta diferenciación de jobs locales o jobs remoto nos da las siguientes ventajas:

- Jobs locales: al recibir un JOB_REQUEST, cada solicitud de recursos a otros nodos se guarda en la tabla como un pedido de recursos para que al recibir una respuesta GRANTED del nodo consultado se guarde en la tabla como recurso reservado, de esta manera, cuando se quiere verificar si un job tiene los recursos necesarios, se verifica que todos los

recursos pedidos por ese job id fueron reservados. Cuando se recibe un JOB_RELEASE, como los recursos reservados son de nodos ajenos, se busca todos los recursos pedidos por ese job (a quién se le pidió y que recurso se pidió) para después enviar su respectivo release.

- Jobs remotos: al garantizar un RESERVE de un nodo remoto, se guarda en la tabla los datos del job en la tabla junto a los recursos que fueron reservados. Al recibir un RELEASE (o el nodo fue limpiado por no haberse anunciado en el tiempo suficiente), se liberan los recursos correspondientes del job (o todos) devolviendolos como recursos disponibles y en caso de que el job después de liberar los recursos no tenga recursos asignados, se borra de la tabla.
- **Recursos:** Cada recurso (cpu, memoria, gpu), guarda la cantidad del recurso total que tenga el sistema, la cantidad disponible que no haya sido reservada por algún job y una cola FIFO de solicitudes pendientes por atender.

Cuando se recibe una solicitud de reserva de algún recurso, si la cantidad disponible del recurso es suficiente y no hay solicitudes pendientes, se descuenta el recurso y se guarda en la tabla de jobs dicha reserva. En otro caso, se encola la solicitud en una cola simple que guarda todos los datos necesarios de la solicitud (quién, qué, cuánto reserva).

2.3. Manejo de Concurrency y Sockets no Bloqueantes

Cada hilo ejecuta una instancia de epoll para el manejo concurrente de solicitudes. Los sockets de escucha, tanto para las conexiones privadas como públicas, se encuentran en modo no bloqueante. Una vez que un socket de escucha acepta a un cliente este se clasifica según si es un Agente C, o un cliente de ERLANG o si es CONEXION_SALIENTE que especificaremos más adelante. Si el cliente es Agente C se parsea la entrada y se actúa según el comando recibido, actualizando las estructuras de datos pertinentes como los recursos del nodo, la tabla de jobs, etc. Para clientes ERLANG el proceso es análogo, salvo que para el caso de JOB_REQUEST se crea un socket efímero para comunicarse con el nodo correspondiente al que erlang le ha pedido recursos. Este socket se crea en modo de escritura no bloqueante, por lo que se inicializa con la bandera EPOLLOUT y EPOLLONESHOT y se agrega a la instancia de epoll para que el mismo nos avise cuando haya establecido la conexión y sea posible escribir. Una vez que se puede escribir, se envía el mensaje de RESERVE al nodo correspondiente y se modifica el socket en el epoll esta vez con la opción EPOLLIN, ya que esperamos que el nodo nos responda GRANTED o DENIED. En caso que nos responda GRANTED entonces se usará el socket para enviar finalmente RELEASE y se eliminará el socket.

Para evitar la fragmentación o truncamiento de mensajes, se utiliza como convención que los mensajes se terminen con un salto de línea.

3. Protocolos de Comunicación y Diagrama de Secuencia

3.1. Interfaz Local (Erlang ↔ C)

Para la comunicación Erlang ↔ C , se hace uso de un conjunto de comandos con el objetivo de la comunicación entre los agentes, que se lleva a cabo mediante el uso de `gen_tcp:connect` y `gen_tcp:send`

- GET NODES se utiliza para que el agente C envíe hacia el `job_scheduler` (a través del manager de entrada y salida de erlang) la lista de nodos con formato "NODES ... \n"
- JOB_REQUEST se utiliza para solicitar un conjunto de recursos de los nodos almacenados en la tabla. Devuelve al Erlang JOB_GRANTED en caso de poder reservar los recursos, JOB_DENIED en caso de no poder hacer la reserva o JOB_TIMEOUT en caso de que la reserva se haya agregado a la cola de espera
- JOB_RELEASE se utiliza para liberar todos los recursos que son retenidos actualmente por un job.

3.2. Protocolo entre Nodos C (Red Pública)

Los cuatro comandos clave operan bajo el siguiente formato y semántica:

- RESERVE <jobId> <tipo_recurso> <cantidad>\n: Mensaje enviado por el agente local hacia un nodo destino para solicitar y bloquear de forma aislada una cantidad específica de un recurso unitario (CPU, MEM o GPU).
- GRANTED <jobId> <tipo_recurso>\n: Respuesta asíncrona enviada por el nodo remoto de vuelta al agente local, confirmando que el recurso solicitado ha sido reservado con éxito en su entorno para ese identificador de trabajo (jobId).
- DENIED <jobId> <tipo_recurso>\n: Respuesta que indica que el nodo remoto no pudo satisfacer la solicitud de reserva (por ejemplo, debido a una condición de carrera, cambios en la disponibilidad o falta de capacidad de último momento).
- RELEASE <jobId> <tipo_recurso>\n: Mensaje utilizado para liberar recursos previamente reservados en un nodo, ya sea porque el trabajo finalizó su ejecución con éxito o porque la transacción global falló y requiere un *rollback* de los fragmentos ya aprobados.

3.3. Diagrama de Secuencia General

A continuación, se presenta el diagrama que ilustra el ciclo de vida completo de una solicitud de recursos, desde su origen en el planificador Erlang hasta la negociación entre los agentes C remotos.

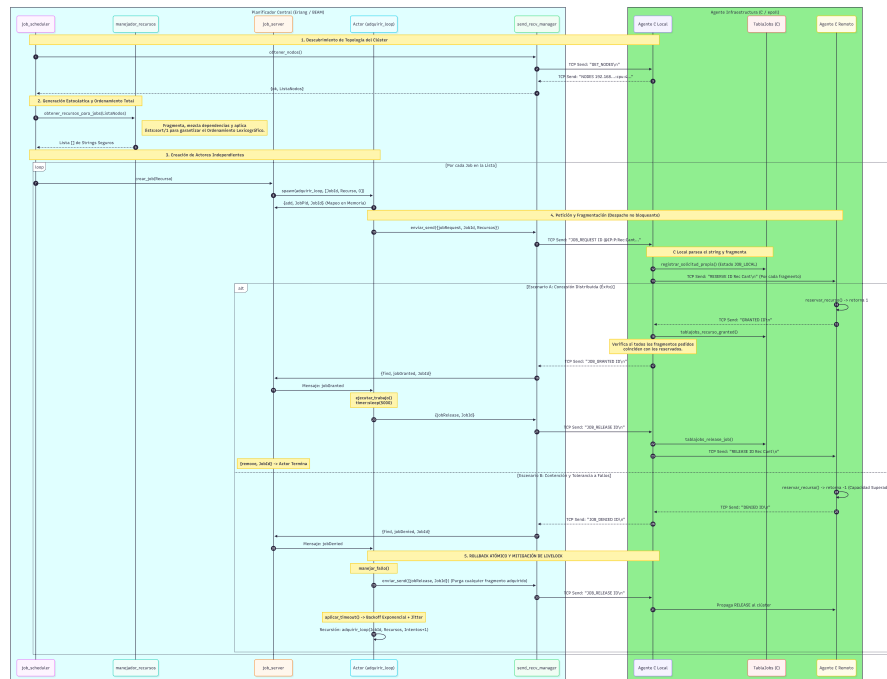


Figura 1: Diagrama de secuencia de la comunicación local y remota.

4. Estrategia contra Deadlocks Distribuidos

Para la estrategia anti-deadlock habia dos metodos principales:

Prevención: Garantizar que el estado de espera circular sea lógicamente inalcanzable. Para esto, es crucial establecer las invariantes del sistema relativas a las precondiciones de asignación antes de iniciar cualquier ciclo de petición de recursos. Un enfoque clásico es imponer un orden total estricto en la adquisición de recursos

Detección y Recuperación: Permitir que los recursos se soliciten libremente, pero mantener un "grafo de espera"(Wait-For Graph) para modelar las dependencias. Si el grafo forma un ciclo, se viola la invariante de progreso, y el planificador debe abortar (hacer *rollback* enviando JOB_RELEASE) y encolar el job causante para romper el ciclo.

Dentro de las 4 requisitos los cuales son necesarios para un deadlock, el más sencillo para nosotros de evitar/corregir era el de la espera circular. Por lo tanto decidimos por el metodo de prevenir el deadlock estableciendo orden dentro de las peticiones.

Si lográbamos ordenar todos los recursos del sistema (por ejemplo, alfabéticamente o por ID) y **obligar** a todos los procesos a pedir los recursos estrictamente en ese orden, sería imposible llegar a un deadlock.

El sistema anula la posibilidad de una Espera Circular (Circular Wait) en el clúster. Antes de serializar la petición hacia la red, el módulo de generación (job_generator) formatea y ordena lexicográficamente los tokens de recursos mediante `lists:sort/1`.

Entonces, al imponer un orden total general, se asegura que si dos Jobs indepen-

dientes requieren el Nodo A y el Nodo B, ambos intentarán adquirir estrictamente el Nodo A antes que el Nodo B. Esto elimina por completo que se crucen las peticiones y por lo tanto, no se bloqueen.

5. Referencias Bibliográficas

Referencias

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- [2] *epoll(7) — Linux manual page*. Disponible en: <https://man7.org/linux/man-pages/man7/epoll.7.html>
- [3] *Erlang/OTP Documentation — gen_tcp*. Disponible en: https://www.erlang.org/doc/man/gen_tcp.html